

M18 / PROTECTED ANSWERS

M18 Explained Project Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

M18 worked project reviews 3

Running the protected reference 3

P01 - explain every delivered row 3

P02 - preserve invoice grain 3

P03 - separate correctness from performance 3

CAP - explain the revenue gap and inventory meaning 4

Prove the publication boundary 4

Complete executable reference 4

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

M18 worked project reviews

Open this after your own attempt. The Python reference uses only the standard library and the supplied fixtures. It proves these small business contracts and a single-writer publication design; it does not substitute for your PostgreSQL/dbt, Spark/Delta, Kafka, Airflow or Fabric implementation.

Running the protected reference

From 17_M18, run:

```
python PROTECTED_OPEN_AFTER_ATTEMPT/reference/project_reference.py P01 --output review-output/P01
```

Replace P01 with P02, P03 or CAP to check the other fixture. Use a fresh output directory for a clean run. The reference writes immutable raw acquisitions and a generation selected by CURRENT.json. Readers use only the generation named in CURRENT.json.

P01 - explain every delivered row

Day 1 has 12 delivered rows: 9 accepted current orders, 1 identical duplicate and 2 quarantined records. Paid revenue is 2150.00. Day 2 contains one update and two inserts, so the target grows from 9 to 11 orders rather than 12. Paid revenue becomes 2940.00. Applying the same delivered batch again does not add its revenue a second time. The update replaces the order's prior state. Customer C999 is unknown and a negative quantity is invalid; neither belongs in paid revenue.

P02 - preserve invoice grain

There are 12 invoice facts and 10 paid invoices. Join invoice.subscription_id to subscription.subscription_id before reaching the customer. C001 has two subscriptions, so joining through customer alone can multiply a fact. The six paid mart rows are August ENT 498, MID 148, SMB 148 and September ENT 249, MID 49, SMB 148: total 1240.00. The base input has no reliable update field; a full rebuild is the honest baseline.

P03 - separate correctness from performance

The 60 valid events have 60 unique event IDs, 20 purchases and 12 date/channel Gold rows. Daily purchase totals are 300, 270 and 315: total 885. An exact event replay does not change those controls. The malformed timestamp/NaN fixture adds one rejected record. device_type is an additive optional raw attribute: the reference's declared Silver projection ignores it, so E0001 remains the same business event. A changed amount under E0001 would still be a conflict. The tiny input establishes a result contract, not distributed throughput.

CAP - explain the revenue gap and inventory meaning

The 12 orders total 1380.00. Eleven captured payments total 1240.00. Order OR009 has a declined payment for 140.00, explaining the gap. Keep it visible with a left join; an inner join would conceal an unpaid order. Two captured events with distinct payment IDs for one order require a business duplicate check. Inventory totals are changes: P01 -5, P02 -7, P03 -9, P04 -11 and P05 -4. Without opening stock these cannot be labelled ending inventory balances. General serving output excludes customer email.

Prove the publication boundary

Run a project successfully and save CURRENT.json. Run it again with --fail-before-publish. The command deliberately fails after preparing a generation, while CURRENT.json still names the last successful generation. Rerun without that flag: the result converges. This design assumes one writer and a local filesystem supporting atomic file replacement. It is not a distributed transaction. For your native project, demonstrate the equivalent boundary using its actual storage and execution engine.

Complete executable reference

Read the function for one project before the publication function. The same current code is supplied in reference/project_reference.py.

```
"""Protected local correctness reference for the four M18 fixture contracts.
```

```
This provides executable business controls and crash/replay evidence. It does not
replace the learner's PostgreSQL/dbt, Spark/Delta, Airflow or Fabric deployment.
Run from any folder: python project_reference.py P01 --output /your/empty/folder
Python standard library only. No input file is edited. One writer per output.
"""
```

```
from __future__ import annotations
import argparse
import csv
import hashlib
import json
import os
import tempfile
from collections import Counter, defaultdict
from datetime import datetime
from decimal import Decimal
from pathlib import Path
```

```
ROOT = Path(__file__).resolve().parents[2] / "practice"
PROJECTS = {
    "P01": "P01_batch_pipeline",
    "P02": "P02_analytics_engineering",
    "P03": "P03_scalable_lakehouse",
    "CAP": "CAP_production_capstone",
}
```

```
def read(path):
    if path.suffix == ".csv":
        with path.open(newline="", encoding="utf-8") as h:
            return list(csv.DictReader(h))
    return [json.loads(line) for line in path.read_text().splitlines() if line.strip()]
```

```
def number(value):
```

```

if isinstance(value, bool):
    raise ValueError("boolean is not money")
n = Decimal(str(value))
if not n.is_finite() or n < 0 or n.as_tuple().exponent < -2:
    raise ValueError(
        "money must be finite, nonnegative, with at most two decimal places"
    )
return n

def timestamp(value):
    if not isinstance(value, str) or not value.endswith("Z"):
        raise ValueError("timestamp must end in Z")
    return datetime.fromisoformat(value[:-1] + "+00:00")

def unique(rows, key):
    result = {}
    for row in rows:
        k = row.get(key)
        if not k or k in result:
            raise ValueError("missing or duplicate " + key + ": " + str(k))
        result[k] = row
    return result

def atomic(path, obj):
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp = tempfile.mkstemp(dir=path.parent, prefix=path.name + ".")
    try:
        with os.fdopen(fd, "w") as h:
            json.dump(obj, h, sort_keys=True, indent=2, allow_nan=False)
            h.flush()
            os.fsync(h.fileno())
        os.replace(tmp, path)
    except BaseException:
        Path(tmp).unlink(missing_ok=True)
        raise

def p01(base):
    customers = unique(read(base / "customers.csv"), "customer_id")
    products = unique(read(base / "products.csv"), "product_id")
    current = {}
    quarantine = []
    metrics = []
    for filename in ("orders_2026-09-01.csv", "orders_2026-09-02.csv"):
        rows = read(base / filename)
        accepted = duplicates = 0
        for line, r in enumerate(rows, 2):
            try:
                timestamp(r["updated_at"])
                if not r["order_id"].strip():
                    raise ValueError("missing_order_id")
                qty = Decimal(r["qty"])
                if not qty.is_finite() or qty <= 0 or qty != qty.to_integral_value():
                    raise ValueError("invalid_qty")
                if r["customer_id"] not in customers:
                    raise ValueError("unknown_customer")
                if r["product_id"] not in products:
                    raise ValueError("unknown_product")
                if r["status"] not in {"PAID", "PENDING", "CANCELLED"}:
                    raise ValueError("invalid_status")
                number(products[r["product_id"]]["unit_price"])

```

```

except Exception as e:
    quarantine.append(
        {"file": filename, "line": line, "reason": str(e), "raw": r}
    )
    continue
old = current.get(r["order_id"])
if old and timestamp(r["updated_at"]) == timestamp(old["updated_at"]):
    if old != r:
        raise ValueError("equal_version_conflict:" + r["order_id"])
    duplicates += 1
    continue
if old and timestamp(r["updated_at"]) < timestamp(old["updated_at"]):
    raise ValueError("unexpected stale row in supplied ordered fixture")
current[r["order_id"]] = r
accepted += 1
rejected = sum(q["file"] == filename for q in quarantine)
assert len(rows) == accepted + duplicates + rejected
paid = sum(
    (
        Decimal(r["qty"]) * number(products[r["product_id"]]["unit_price"])
        for r in current.values()
        if r["status"] == "PAID"
    ),
    Decimal(0),
)
metrics.append(
    {
        "file": filename,
        "source_rows": len(rows),
        "accepted_changes": accepted,
        "duplicate_rows": duplicates,
        "quarantined": rejected,
        "target_orders": len(current),
        "paid_revenue": str(paid),
    }
)
return {"silver": list(current.values()), "quarantine": quarantine, "gold": metrics}

```

```

def p02(base):
    customers = unique(read(base / "customers.csv"), "customer_id")
    plans = unique(read(base / "plans.csv"), "plan_id")
    subscriptions = unique(read(base / "subscriptions.csv"), "subscription_id")
    invoices = unique(read(base / "invoices.csv"), "invoice_id")
    for r in subscriptions.values():
        if r["customer_id"] not in customers or r["plan_id"] not in plans:
            raise ValueError("subscription relationship failure")
    facts = []
    amounts = defaultdict(Decimal)
    counts = Counter()
    for r in invoices.values():
        if r["subscription_id"] not in subscriptions:
            raise ValueError("orphan_invoice")
        if r["status"] not in {"PAID", "OPEN"}:
            raise ValueError("invalid_invoice_status")
        sub = subscriptions[r["subscription_id"]]
        customer = customers[sub["customer_id"]]
        amount = number(r["amount"])
        fact = {
            **r,
            "customer_id": sub["customer_id"],
            "segment": customer["segment"],
            "plan_id": sub["plan_id"],
        }

```

```

facts.append(fact)
if r["status"] == "PAID":
    key = (r["invoice_date"][:7], customer["segment"])
    amounts[key] += amount
    counts[key] += 1
marts = [
    {
        "month": m,
        "segment": s,
        "paid_invoices": counts[(m, s)],
        "paid_revenue": str(v),
    }
    for (m, s), v in sorted(amounts.items())
]
assert len(facts) == len(invoices)
return {"silver": facts, "quarantine": [], "gold": marts}

```

```

def p03(base):
    products = unique(read(base / "products.csv"), "product_id")
    campaigns = unique(read(base / "campaigns.csv"), "campaign_id")
    events = read(base / "events.jsonl")
    seen = {}
    bad = []
    facts = []
    totals = defaultdict(Decimal)
    for n, r in enumerate(events, 1):
        try:
            timestamp(r["event_time"])
            amount = number(r["amount"])
            if r["product_id"] not in products or r["campaign_id"] not in campaigns:
                raise ValueError("unknown_reference")
            if r["event_type"] not in {"view", "add_to_cart", "purchase"}:
                raise ValueError("unknown_event_type")
            if not r["event_id"] or not r["user_id"]:
                raise ValueError("missing_identity")
            # This Silver schema intentionally ignores optional additive raw fields.
            r = {
                k: r[k]
                for k in (
                    "event_id",
                    "event_time",
                    "user_id",
                    "event_type",
                    "product_id",
                    "campaign_id",
                    "amount",
                )
            }
        except Exception as e:
            bad.append({"line": n, "reason": str(e), "raw": r})
            continue
        if r["event_id"] in seen:
            if seen[r["event_id"]] != r:
                raise ValueError("event_identity_conflict")
            continue
        seen[r["event_id"]] = r
        fact = {
            **r,
            "event_date": r["event_time"][:10],
            "category": products[r["product_id"]]["category"],
            "channel": campaigns[r["campaign_id"]]["channel"],
        }
        facts.append(fact)

```

```

        if r["event_type"] == "purchase":
            totals[fact["event_date"], fact["channel"]] += amount
    marts = [
        {"event_date": d, "channel": c, "purchase_amount": str(v)}
        for (d, c), v in sorted(totals.items())
    ]
    return {"silver": facts, "quarantine": bad, "gold": marts}

def cap(base):
    customers = unique(read(base / "customers.csv"), "customer_id")
    orders = unique(read(base / "orders.csv"), "order_id")
    payments = unique(read(base / "payments.jsonl"), "payment_id")
    inventory = unique(read(base / "inventory_cdc.jsonl"), "change_id")
    unique(
        list(inventory.values()), "source_seq"
    ) # one global sequence in this fixture
    captured = defaultdict(Decimal)
    capture_count = Counter()
    delta = defaultdict(int)
    for p in payments.values():
        timestamp(p["paid_at"])
        amount = number(p["amount"])
        if p["order_id"] not in orders:
            raise ValueError("orphan_payment")
        if p["status"] not in {"CAPTURED", "DECLINED"}:
            raise ValueError("invalid_payment_status")
        if p["status"] == "CAPTURED":
            captured[p["order_id"]] += amount
            capture_count[p["order_id"]] += 1
    if any(n > 1 for n in capture_count.values()):
        raise ValueError("multiple_captures_require_business_review")
    facts = []
    for order_id, r in orders.items():
        amount = number(r["amount"])
        timestamp(r["order_ts"])
        if r["customer_id"] not in customers:
            raise ValueError("unknown_customer")
        if captured[order_id] > amount:
            raise ValueError("over_capture")
        # General serving output deliberately excludes raw customer pii_email.
        facts.append(
            {
                "order_id": order_id,
                "region": customers[r["customer_id"]]["region"],
                "order_amount": str(amount),
                "captured_amount": str(captured[order_id]),
            }
        )
    for r in inventory.values():
        timestamp(r["changed_at"])
        if type(r["delta_qty"]) is not int:
            raise ValueError("delta_qty must be integer")
        delta[r["product_id"]] += r["delta_qty"]
    # CDC deltas are additive events, not current stock balances. Initial stock is not supplied.
    gold = {
        "orders": len(facts),
        "order_amount": str(sum(number(x["order_amount"]) for x in facts)),
        "captured_amount": str(sum(number(x["captured_amount"]) for x in facts)),
        "unpaid_order_ids": sorted(k for k in orders if not captured[k]),
        "inventory_change_totals": dict(sorted(delta.items())),
    }
    return {"silver": facts, "quarantine": [], "gold": gold}

```

```

def run(project, output, source=None, fail_before_publish=False):
    source = Path(source) if source else ROOT / PROJECTS[project] / "data/raw"
    output = Path(output)
    output.mkdir(parents=True, exist_ok=True)
    digest = hashlib.sha256()
    for p in sorted(source.iterdir()):
        if p.is_file():
            digest.update(p.name.encode())
            digest.update(p.read_bytes())
    identity = digest.hexdigest()
    snapshot = output / "raw" / identity
    snapshot.mkdir(parents=True, exist_ok=True)
    for p in sorted(source.iterdir()):
        if not p.is_file():
            continue
        dest = snapshot / p.name
        if dest.exists():
            if dest.read_bytes() != p.read_bytes():
                raise ValueError("immutable_raw_collision")
        else:
            with dest.open("xb") as h:
                h.write(p.read_bytes())
                h.flush()
                os.fsync(h.fileno())
    result = {"P01": p01, "P02": p02, "P03": p03, "CAP": cap}[project](snapshot)
    # One immutable generation contains all outputs. Only CURRENT.json publishes it.
    result_id = hashlib.sha256(json.dumps(result, sort_keys=True).encode()).hexdigest()
    gen = output / "generations" / result_id
    gen.mkdir(parents=True, exist_ok=True)
    for name, obj in result.items():
        atomic(gen / (name + ".json"), obj)
    if fail_before_publish:
        raise RuntimeError("deliberate crash before publication; CURRENT unchanged")
    atomic(
        output / "CURRENT.json", {"source_sha256": identity, "generation": result_id}
    )
    return result

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("project", choices=PROJECTS)
    ap.add_argument("--output", type=Path, required=True)
    ap.add_argument("--source", type=Path)
    ap.add_argument("--fail-before-publish", action="store_true")
    a = ap.parse_args()
    result = run(a.project, a.output, a.source, a.fail_before_publish)
    print(
        json.dumps(
            {
                "project": a.project,
                "silver_rows": len(result["silver"]),
                "quarantine_rows": len(result["quarantine"]),
                "gold": result["gold"],
            },
            indent=2,
        )
    )

```